SERVICIO NACIONAL DE APRENDIZAJE – SENA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE – ADSO

GUÍA DE ESTRUCTURA Y EVIDENCIAS DEL REPOSITORIO

Organización, gobierno, trazabilidad, automatización y paquete final de entrega

Campo	Información
Nombre del proyecto	Sistema de gestión para gimnasio Zona Fit Evolution
Centro de formación	Centro de Tecnología de la Manufactura Avanzada
Ficha	3223873
Repositorio principal	https://github.com/zonafitevolutionSENA/ZonaFitEvolution03
Rama de entrega	main
Etiqueta final	v1.0.0 (pendiente)
Equipo responsable	Juan Diego Mejía Llano, Liliana Sofía Panesso Méndez y Maileth Begambre Movilla
Versión del documento	0.1
Fecha	03/11/2026
Estado	Borrador

Finalidad
Establecer una estructura mínima, consistente y verificable para que el repositorio sea la fuente viva del producto, conserve evidencia del proceso y permita reproducir la entrega sin depender del equipo autor.

Control documental

Versión	Fecha	Cambio	Elaboró	Revisó	Aprobó	Estado
0.1	03/11/2026	Estado actual del repositorio y brechas frente a la guía	Juan Diego Mejía Llano			Borrador

Aprobaciones

Rol	Nombre	Aspecto validado	Fecha	Firma / aprobación
Líder técnico	Liliana Sofía Panesso Méndez	Estructura, scripts y reproducibilidad		
Responsable de calidad		Evidencias y trazabilidad		
Instructor / evaluador		Suficiencia pedagógica y técnica		
Representante del equipo	Juan Diego Mejía Llano	Correspondencia con la entrega		

1. Propósito, alcance y principios

Esta guía define qué debe contener el repositorio, cómo debe organizarse, qué artefactos son obligatorios y cómo se conserva evidencia del desarrollo. Complementa los documentos del proyecto y evita convertir Word en la única fuente de información técnica.

1.1 Principios

- El repositorio es la fuente de verdad del producto ejecutable.
- La documentación debe mantenerse cerca del código que describe.
- Todo cambio importante debe dejar evidencia identificable.
- Los secretos y datos personales no pertenecen al repositorio.
- La instalación, pruebas y construcción deben ser reproducibles.
- La estructura debe ser comprensible para una persona externa al equipo.
- La automatización debe comprobar calidad, no sustituir la comprensión.

Criterio de cierre
Un repositorio no se considera listo porque “funciona en el equipo del autor”. Debe poder clonarse, configurarse, migrarse, probarse y ejecutarse siguiendo instrucciones controladas.

2. Estructura recomendada del repositorio

2.1 Árbol de referencia

```
ZonaFitEvolution03/
├── README.md, .gitignore
```

├── AgenteLector/ (index.js, .env.example, package.json: agente del lector AS608 para la nube)

├── BackendZonafitevolution/ (database/init.sql, scripts/crear-admin.js, src/, .env.example, index.js, package.json)

├── FrontendZonafitevolution/ (src/pages, components, services, utils; index.html; vite.config.js; package.json)

└── docs/analisis/

Faltan: /tests, /docs/adr, /docs/architecture, /docs/api, /docs/testing, /docs/evidence y /.github/workflows.

2.2 Responsabilidad por directorio

Ruta	Responsabilidad	Contenido permitido	Contenido no permitido
/src	Backend y frontend (BackendZonafitevolution /src, FrontendZonafitevolution /src)	Módulos, capas, componentes	Backups, secretos, archivos temporales
/tests	No existe (TT-007)	Unitarias, integración, E2E	Evidencia manual sin contexto
/docs	Solo docs/analisis	ADR, diagramas, API, guías	Copias desactualizadas sin versión
/migrations	No existe (init.sql)	Migraciones ordenadas y reversibles	SQL manual sin control
/scripts	BackendZonafitevolution/ scripts	Setup, build, seed, backup	Scripts destructivos sin advertencia
/infra	No existe	Contenedores, proxy, manifests	Credenciales reales
/.github	No existe (TT-008)	Workflows, PR, issues	Tokens o archivos generados
/public o equivalente	FrontendZonafitevolution /public	Imágenes y archivos públicos	Datos confidenciales

3. Archivos raíz obligatorios

Archivo	Obligatorio	Propósito	Criterio mínimo
README.md	Sí – existe	Entrada principal al proyecto	Descripción, instalación, ejecución, pruebas y enlaces
LICENSE	No existe	Condiciones de uso y distribución	Licencia compatible y atribuciones
CHANGELOG.md	No existe	Historial legible de cambios	Versiones, fecha y categorías de cambio
.gitignore	Sí – existe	Excluir archivos no versionables	Secretos, dependencias, builds y temporales
.env.example	Sí – existe (backend)	Declarar variables requeridas	Nombres completos y valores ficticios
Lockfile	Sí – existe (backend y	Reproducibilidad de	Generado por el gestor y

Archivo	Obligatorio	Propósito	Criterio mínimo
	frontend)	dependencias	versionado
CONTRIBUTING.md	No existe	Reglas de colaboración	Ramas, commits, PR y revisión
SECURITY.md	No existe	Reporte de vulnerabilidades	Canal y tratamiento responsable
CODEOWNERS	No existe	Responsabilidad de revisión	Propietarios por rutas críticas

4. Plantilla mínima de README.md

Sección	Contenido esperado	Evidencia de calidad
Título y estado	Nombre, propósito y estado del proyecto	Se identifica la versión actual
Problema y solución	Necesidad abordada y propuesta	No confunde problema con tecnología
Características	Funciones implementadas	Coinciden con el producto
Tecnologías	Stack y versiones relevantes	Se derivan de manifiestos reales
Requisitos previos	Software, cuentas y puertos	Permiten preparar el ambiente
Instalación	Comandos secuenciales	Probados en ambiente limpio
Configuración	Variables y archivos de ejemplo	Sin secretos reales
Migraciones y semillas	Creación de datos	Reproducible desde cero
Ejecución	Desarrollo y producción	Comandos exactos
Pruebas	Suites, cobertura y comandos	Resultados verificables
Despliegue	Procedimiento o enlace al manual	Versión y ambiente identificables
Estructura	Directorios principales	Facilita mantenimiento
Autores y contribuciones	Equipo y responsabilidades	Coherente con historial Git
Licencia	Licencia y terceros	Atribuciones completas

5. Versionamiento y CHANGELOG

5.1 Convención de versiones

Elemento	Convención	Ejemplo
Versión de producto	SemVer u otra estrategia definida	1.4.0
Etiqueta Git	v<versión>	v1.4.0
Versión documental	Compatible con la versión del producto	1.4
Imagen de contenedor	<producto>:<versión>	inventario:1.4.0
Migración	Fecha/secuencia + descripción	20260721_add_audit_log

5.2 Categorías del CHANGELOG

- Added: funcionalidades nuevas.
- Changed: cambios funcionales o técnicos.
- Deprecated: elementos que serán retirados.
- Removed: funcionalidades eliminadas.
- Fixed: errores corregidos.
- Security: correcciones o cambios de seguridad.

5.3 Registro de liberaciones

Versión	Fecha	Commit / etiqueta	Ambiente	Responsable	Notas
					Sin etiquetas; última integración en main el 26/10/2026 (docs)

6. Gobierno de Git y colaboración

6.1 Estrategia de ramas

Rama	Propósito	Regla de protección	Integración
main	Versión estable	PR y pipeline obligatorios	Pull Request desde develop
develop	Integración de cada sprint	PR y pruebas	Pull Request desde feature/*
feature/*	Cada historia o tarea	Una necesidad coherente por rama	Se crea desde develop; no se borra al integrar
fix/*		Prueba de regresión	
hotfix/*		Revisión prioritaria	
release/*		Congelación y validación	

6.2 Convención de commits

Tipo	Uso	Ejemplo
feat	Nueva capacidad	feat(loans): add return validation
fix	Corrección	fix(auth): reject expired token
docs	Documentación	docs(readme): add deployment steps
test	Pruebas	test(users): cover duplicate email
refactor	Cambio interno	refactor(api): centralize errors
chore	Mantenimiento	chore(deps): update lockfile
ci	Pipeline	ci: add security audit

6.3 Revisión por pull request

Campo de PR	Contenido obligatorio
Resumen	Qué cambia y por qué
Requisito relacionado	RF, RNF, HU, incidencia o tarea
Tipo de cambio	Funcional, corrección, técnico o documental
Pruebas	Comandos y resultados
Capturas / evidencia	Cuando existe cambio visible
Riesgos	Impactos y compatibilidad
Migraciones	Sí/No y procedimiento
Checklist	Lint, pruebas, seguridad y documentación

7. Plantillas de issues y trabajo

Plantilla	Campos mínimos	Cuándo usar
Historia / funcionalidad	Objetivo, criterios, prioridad, dependencias	Nueva capacidad
Defecto	Pasos, esperado, real, ambiente, severidad	Comportamiento incorrecto
Tarea técnica	Motivo, alcance, riesgo, aceptación	Refactor, infraestructura o deuda
Incidente	Impacto, hora, versión, contención	Fallo operativo
Solicitud de cambio	Origen, impacto, aprobación, trazabilidad	Cambio de alcance

7.1 Definición de listo

- Criterios de aceptación cumplidos.
- Código revisado e integrado.
- Pruebas nuevas o actualizadas.
- Pipeline aprobado.
- Documentación afectada actualizada.
- Sin secretos ni datos sensibles.
- Evidencia enlazada.
- Deuda o limitación registrada cuando aplique.

8. Evidencias y trazabilidad

8.1 Convención de evidencias

Elemento	Convención	Ejemplo
Evidencia	EV-###	EV-014
Entrega	ENTG-##	ENTG-03
Caso de prueba	CP-##	CP-21

Elemento	Convención	Ejemplo
Decisión	ADR-###	ADR-005
Incidente	INC-##	INC-04
Despliegue	DEP-##	DEP-02

8.2 Matriz de evidencia

ID	Elemento trazado	Tipo de evidencia	Ubicación	Versión	Responsable	Estado
EV-001	Commits por integrante	Historial de Git	https://github.com/zonafitevolutionSENA/ZonaFitEvolution03	main	Juan Diego Mejía Llano	Disponible
EV-002	Backlog e historias	Tablero	Tablero de Jira del proyecto SCRUM		Liliana Sofía Panesso Méndez	Disponible
EV-003	Ejecución de casos CP-001 a CP-021	Reporte de pruebas				Pendiente
EV-004	Despliegue v1.0.0	URL y prueba de humo			Liliana Sofía Panesso Méndez	Pendiente

Evidencia válida
Una captura aislada no demuestra por sí sola que una función esté implementada correctamente. Debe conservar contexto, versión y relación con requisito, prueba o entrega.

9. Documentación técnica viva en /docs

Carpeta / archivo	Contenido	Formato recomendado
/docs/adr	Decisiones arquitectónicas	ADR-###-titulo.md
/docs/architecture	Contexto, contenedores, componentes	Mermaid, PlantUML o imágenes fuente
/docs/api	Contratos y ejemplos	OpenAPI, colecciones y Markdown
/docs/data	Modelo y diccionario	Diagramas y tablas versionadas
/docs/security	Modelo de amenazas y controles	Markdown
/docs/testing	Estrategia y ejecución	Markdown y reportes enlazados
/docs/operations	Runbooks y procedimientos	Markdown
/docs/evidence	Índice de evidencias	Markdown, no archivos pesados innecesarios

9.1 Plantilla ADR

Campo	Contenido
Título	ADR-###: decisión concreta
Estado	Propuesto / Aceptado / Reemplazado / Rechazado
Contexto	Problema y fuerzas que influyen
Opciones	Alternativas consideradas
Decisión	Opción seleccionada
Consecuencias	Beneficios, costos y riesgos
Trazabilidad	Requisitos, componentes y entregas relacionadas

10. API, contratos y colecciones

Artefacto	Contenido mínimo	Validación
OpenAPI	Rutas, métodos, esquemas, seguridad y errores	Coincide con API desplegada
Colección de pruebas	Variables, autenticación y ejemplos	Ejecutable sin secretos reales
Ejemplos de payload	Casos válidos e inválidos	No contienen datos sensibles
Catálogo de errores	Código, significado y acción	Consistente con implementación
Versionamiento de API	Política de compatibilidad	Cambios incompatibles identificados

11. Configuración, secretos y archivos ignorados

11.1 .env.example

Regla	Aplicación
Cobertura	Debe declarar todas las variables necesarias
Valores	Usar ejemplos ficticios y seguros
Descripción	Comentar propósito, formato y obligatoriedad
Separación	Diferenciar desarrollo, pruebas y producción
Validación	La aplicación debe fallar claramente si falta una variable crítica
Actualización	Todo cambio de configuración debe actualizar el ejemplo

11.2 Contenido que nunca debe versionarse

- Archivos .env reales.
- Llaves privadas, tokens y certificados secretos.
- Copias de bases de datos con información real.
- Credenciales incrustadas en código o documentación.

- Logs con información personal o tokens.
- Archivos de usuario no autorizados.
- Backups, builds y dependencias regenerables, salvo decisión justificada.

12. CI/CD y evidencias automáticas

Etapa	Control mínimo	Evidencia
Instalación	Dependencias desde lockfile	Log del job
Calidad estática	Lint y formato	Resultado del pipeline
Pruebas	Unitarias e integración	Reporte y estado
Cobertura	Generación de reporte	Artefacto o resumen
Build	Compilación o empaquetado	Artefacto versionado
Seguridad	Auditoría de dependencias y secretos	Resultado sin críticos
Migraciones	Validación controlada	Log de esquema
Despliegue	Publicación autorizada	Versión, ambiente y hora
Smoke test	Verificación posterior	Resultado y evidencia

12.1 Reglas del pipeline

- Debe ejecutarse automáticamente en pull requests y ramas definidas.
- Un control crítico fallido debe bloquear la integración.
- No debe imprimir secretos en logs.
- Los artefactos deben asociarse al commit o etiqueta.
- Los jobs deben ser reproducibles y evitar pasos manuales ocultos.

13. Licencias, autoría y uso de IA

13.1 Dependencias y terceros

Elemento	Fuente	Licencia	Compatibilidad revisada	Atribución requerida

13.2 Declaración de uso de IA

Campo	Contenido
Herramienta	Nombre y modalidad
Uso realizado	Código, explicación, pruebas, diseño u otro
Partes afectadas	Archivos o componentes

Campo	Contenido
Validación humana	Cómo se revisó, probó y adaptó
Limitaciones	Errores detectados y correcciones
Responsabilidad	Integrantes responsables de comprender y sustentar

Principio de autoría
Declarar IA no invalida el trabajo. Lo invalida entregar contenido que el equipo no comprende, no verificó o no puede sustentar.

14. Paquete final de entrega

Elemento	Formato / ubicación	Obligatorio	Verificación
Repositorio y etiqueta final	https://github.com/zonafitevolutionSENA/ZonaFitEvolution03 (etiqueta v1.0.0 pendiente)	Sí	Clonable y accesible
Código fuente	BackendZonafitevolution, FrontendZonafitevolution	Sí	Sin secretos ni temporales
README y CHANGELOG	README.md (CHANGELOG no existe)	Sí	Actualizados
Migraciones y semillas	database/init.sql, scripts/crear-admin.js	Sí	Reconstruyen datos
Pruebas y reportes	No existen	Sí	Ejecutables
Documentación API	Tablas de endpoints en el README	Según proyecto	Actualizada
Documentación del proyecto	documentos_proyecto	Sí	Versiones finales
Manual técnico	MAN-01	Sí	Procedimientos verificados
Manual de usuario	MAN-02	Sí	Interfaz final
Artefactos de despliegue		Según proyecto	Versionados
Licencia y atribuciones		Sí	Completas
Declaración de IA		Sí	Firmada o aprobada

14.1 Estructura sugerida del paquete

[Defina la estructura del ZIP o carpeta final sin duplicar innecesariamente lo que ya está en el repositorio]

15. Lista de verificación final del repositorio

N.°	Verificación	Cumple	Evidencia	Observaciones
1	Existe una etiqueta final identificable.	Sí / No / N.A.		
2	README permite instalar y ejecutar el sistema.	Sí / No / N.A.		
3	CHANGELOG refleja la versión entregada.	Sí / No / N.A.		
4	La licencia y atribuciones están presentes.	Sí / No / N.A.		
5	No existen secretos ni datos personales.	Sí / No / N.A.		
6	El lockfile está versionado.	Sí / No / N.A.		
7	Las variables están documentadas en .env.example.	Sí / No / N.A.		
8	Las migraciones reconstruyen el esquema.	Sí / No / N.A.		
9	Las semillas son seguras y reproducibles.	Sí / No / N.A.		
10	Las pruebas se ejecutan con comandos documentados.	Sí / No / N.A.		
11	El pipeline aprueba lint, pruebas y build.	Sí / No / N.A.		
12	La documentación API coincide con la implementación.	Sí / No / N.A.		
13	Los diagramas fuente están disponibles.	Sí / No / N.A.		
14	Las decisiones técnicas relevantes tienen ADR.	Sí / No / N.A.		
15	Las evidencias están trazadas y versionadas.	Sí / No / N.A.		
16	Los pull requests y commits demuestran	Sí / No / N.A.		

N.°	Verificación	Cumple	Evidencia	Observaciones
	el proceso.			
17	La contribución individual es verificable.	Sí / No / N.A.		
18	El despliegue identifica versión y ambiente.	Sí / No / N.A.		
19	El paquete final está completo.	Sí / No / N.A.		
20	Otra persona pudo reproducir la entrega.	Sí / No / N.A.		

16. Cierre de la arquitectura documental

Con esta guía se completa la arquitectura documental del proyecto. Los documentos formales describen decisiones, compromisos, aceptación y cierre; el repositorio conserva el producto vivo, la automatización, la trazabilidad y la evidencia reproducible.

Nivel	Contenido final	Fuente principal
Gobierno institucional	Estándares, estructura y criterios de evaluación	Arquitectura documental y guías
Documentos del proyecto	Formulación, requisitos, diseño, calidad, manuales y cierre	Plantillas diligenciadas
Artefactos vivos	Código, pruebas, migraciones, CI/CD, ADR, API y evidencias	Repositorio
Evaluación	Rúbrica, lista de chequeo y sustentación	Instrumentos de evaluación

16.1 Acta de conformidad del repositorio

Nombre	Rol	Concepto	Firma / aceptación	Fecha
	Líder técnico			
	Instructor			
Juan Diego Mejía Llano	Representante del equipo			

Anexo A. Índice maestro de la suite documental

N.°	Documento / instrumento	Formato sugerido	Estado
1	Arquitectura Documental de Proyectos Software	Word	
2	Formulación y Constitución del Proyecto	Word	

N.°	Documento / instrumento	Formato sugerido	Estado
3	Gestión de Interesados y del Proyecto	Word	
4	Alcance y Especificación de Requisitos	Word	
5	Backlog, Historias de Usuario y Casos de Uso	Word	
6	Matriz Maestra de Trazabilidad	Excel	
7	Diseño UX/UI y Sistema de Diseño	Word	
8	Arquitectura, Datos y Diseño Técnico	Word	
9	Plan de Calidad, Pruebas y Aceptación	Word	
10	Manual Técnico y de Operación	Word	
11	Manual de Usuario	Word	
12	Informe de Cierre y Entrega	Word	
13	Rúbrica Integral de Evaluación	Excel	
14	Lista de Chequeo Final	Excel	
15	Guía de Sustentación Técnica y Demostración	Word	
16	Guía de Estructura y Evidencias del Repositorio	Word	

Anexo B. Errores frecuentes

Error	Consecuencia	Corrección
Subir .env	Exposición de secretos	Revocar, eliminar del historial y usar gestor
Repositorio creado al final	No demuestra proceso	Versionar desde el inicio
README genérico	Instalación no reproducible	Probarlo en ambiente limpio
Commits masivos	Pierde trazabilidad	Hacer cambios pequeños y coherentes
Diagramas sin fuente	No pueden mantenerse	Versionar Mermaid, PlantUML o equivalente
Capturas como única evidencia	No prueban lógica ni calidad	Enlazar pruebas, código y versión
Documentación duplicada	Aparecen contradicciones	Definir una fuente principal
Dependencias sin licencia revisada	Riesgo legal	Registrar licencias y atribuciones
IA no declarada	Dudas de autoría y comprensión	Declarar uso y validación humana